

SOFTWARE CHANGE IMPLEMENTATION REPORT

CR004: Student Input Field Validation for Add and Edit Operations

Field	Details
Change Request ID	CR004
Change Request Title	Student Input Field Validation for Add and Edit Operations
Student Name	Susmitha Vimalarajan
Student ID	JFICT24_30
Implementation Status	Completed
Date	08 October 2026

1. Change Request Summary

1.1 Business Requirement

Administrators currently can enter student information without sufficient validation when adding or editing students. For example, a phone number containing only a few digits may be accepted, and names with fewer than three characters or weak passwords may also be entered. As incorrect or insecure information can affect data quality and account security, the system should validate all student input fields before accepting the data. The system should ensure that student names, registration number, email address, phone number, and password follow the required validation rules

1.2 Existing Behaviour

- StudentFormModal.jsx performs only basic required-field checking.
- A phone number with fewer than 10 digits may be accepted.
- A phone number with more than 10 digits may be entered.
- Non-numeric characters may be entered into the phone number field.
- First name and last name do not have a minimum 3-character requirement.
- Invalid characters may be entered into name fields.
- Email format is not sufficiently validated.
- Registration number does not have comprehensive input validation.
- New student passwords do not enforce a strong password policy.
- Backend validation does not comprehensively validate all student input fields.
- Invalid data may potentially be submitted directly to the backend without passing clientside validation.

1.3 Requested Change

Add comprehensive validation to the student Add and Edit operations.

The implementation must:

- Validate all required student fields before submission.
- Require a valid registration number.
- Require first name and last name to contain at least 3 characters.
- Allow valid alphabetic characters in first name and last name.
- Reject invalid numbers or inappropriate special characters in names.
- Validate the email address using a valid email format.
- Require the phone number to contain exactly 10 digits.
- Allow only numeric characters in the phone number.
- Reject phone numbers containing fewer than 10 digits.
- Reject phone numbers containing more than 10 digits.
- Require a password when creating a new student.
- Require the password to contain at least 8 characters.
- Require at least one uppercase letter in the password. • Require at least one lowercase letter in the password.
- Require at least one number in the password.
- Require at least one special character in the password.
- Display clear validation error messages to administrators.
- Prevent invalid data from being submitted from the frontend.
- Apply server-side validation to prevent client-side validation bypass.
- Preserve existing Add, Edit, and Delete functionality.
- Maintain existing administrator authentication and authorization.
- Continue securely hashing passwords before storing them in the database.

2. Implementation

2.1 Implementation Summary

CR-004 was implemented to improve the validation of student information during the **Add Student** and **Edit Student** operations in the Student Management System.

The main purpose of this change was to prevent administrators from submitting incomplete, invalid, or insecure student information. Both **client-side validation** and **server-side validation** were implemented to ensure that invalid data is rejected before it can be stored in the database.

The implemented validation covers the following student fields:

- Registration Number
- First Name
- Last Name
- Email Address
- Phone Number
- Password

The validation rules ensure that student names contain valid characters and have a minimum length of three characters, email addresses follow a valid format, phone numbers contain exactly ten numeric digits, and new student passwords satisfy the required password strength policy.

Server-side validation was also added to prevent users from bypassing frontend validation by directly sending invalid requests to the API.

Existing student **Add, Edit, Delete, authentication, authorization, database operations, and password hashing mechanisms were preserved.**

2.2 Affected Files

File	Change	Purpose
frontend/src/components/StudentFormModal.jsx	Modified	Implemented client-side validation for student Add and Edit operations and displayed validation error messages.
backend/controllers/studentController.js	Modified	Implemented server-side validation for registration number, names, email, phone number, and password.
Existing student model/API files	No change	Existing database operations and API endpoints were preserved.

Additional files: None

2.3 Technical Changes

Registration Number Validation

The registration number is validated before the Add or Edit operation.

The validation ensures that:

- The registration number is not empty.
- The registration number follows the accepted format.
- Invalid registration numbers are rejected before database operations.

Name Validation

Validation was added for both first name and last name.

The validation ensures that:

- The field is required.
- The name contains at least 3 characters.
- Valid alphabetic characters are accepted.
- Numbers are rejected.
- Inappropriate special characters are rejected.

Email Validation

Email validation was implemented to ensure that the entered email follows a valid email format.

Invalid email addresses are rejected before submission.

Phone Number Validation

Phone validation was implemented to ensure that:

- The phone number is required.
- Only numeric characters are accepted.
- Exactly 10 digits are required.
- Numbers containing fewer than 10 digits are rejected.
- Numbers containing more than 10 digits are rejected.

Strong Password Validation

Strong password validation is applied when creating a new student.

The password must contain:

- At least 8 characters.
- At least one uppercase letter.
- At least one lowercase letter.
- At least one number.
- At least one special character.

The existing password hashing mechanism was preserved, and passwords continue to be securely hashed before being stored.

2.4 Implementation Logic

- Administrator opens the Add or Edit Student form.
- Student information is entered into the form.
- Client-side validation checks the entered information.
- If invalid information is entered, appropriate validation errors are displayed.
- Form submission is prevented when validation fails.
- Valid information is submitted to the existing student API endpoint.
- The backend performs server-side validation again.
- If backend validation fails, the API returns a validation error response.
- If validation succeeds, the existing Add or Edit operation continues.
- During student creation, the password is securely hashed using the existing password hashing mechanism.
- Valid student information is saved successfully.
- Existing Edit and Delete functionality continues to operate using the existing student identity and authorization mechanisms.

2.5 Error and Data Handling

Clear validation error messages are displayed when invalid information is entered.

Examples of validation messages include:

- Registration number is required.
- First name must be at least 3 characters.
- First name contains invalid characters.
- Last name must be at least 3 characters.
- Last name contains invalid characters.
- Please enter a valid email address.
- Phone number must contain exactly 10 digits.
- Phone number must contain numbers only.
- Password must be at least 8 characters.
- Password must contain at least one uppercase letter.
- Password must contain at least one lowercase letter.
- Password must contain at least one number.
- Password must contain at least one special character.

Invalid frontend submissions are prevented from reaching the API.

Server-side validation provides an additional protection layer against direct API requests containing invalid information.

3. Testing & Verification

3.1 Testing Approach

The implementation was verified through manual frontend and backend testing.

Testing Types:

- Functional testing
- Positive testing
- Negative testing
- Validation testing
- UI testing
- Server-side validation testing
- Regression testing
- CRUD verification
- API testing
- Security validation

Environment

Environment: Development

Browser: Chrome

Test Role: Administrator

Backend Testing Tool: Postman

3.2 Test Cases

ID	Test Case	Expected Result	Actual Result	Status
TC01	Submit form with empty required fields	Validation errors are displayed and submission is prevented	Required field errors displayed	Pass
TC02	Enter first name with fewer than 3 characters	First name is rejected	First name rejected	Pass
TC03	Enter last name with fewer than 3 characters	Last name is rejected	Last name rejected	Pass
TC04	Enter numbers in first/last name	Invalid name is rejected	Invalid name rejected	Pass
TC05	Enter inappropriate special characters in name	Invalid name is rejected	Invalid name rejected	Pass
TC06	Enter invalid email address	Email is rejected	Invalid email rejected	Pass
TC07	Enter phone number with fewer than 10 digits	Phone number is rejected	Phone number rejected	Pass
TC08	Enter phone number with more than 10 digits	Phone number is rejected	Phone number rejected	Pass
TC09	Enter non-numeric phone number	Phone number is rejected	Non-numeric phone rejected	Pass
TC10	Enter valid 10-digit phone number	Phone number is accepted	Valid phone accepted	Pass
TC11	Enter weak password	Password is rejected	Weak password rejected	Pass
TC12	Enter password without uppercase letter	Password is rejected	Password rejected	Pass
TC13	Enter password without lowercase letter	Password is rejected	Password rejected	Pass
TC14	Enter password without number	Password is rejected	Password rejected	Pass
TC15	Enter password without special character	Password is rejected	Password rejected	Pass
TC16	Enter valid strong password	Password is accepted	Strong password accepted	Pass
TC17	Create student using valid information	Student is successfully created	Student created successfully	Pass

TC18	Edit student using invalid information	Update is prevented	Invalid update prevented	Pass
TC19	Edit student using valid information	Student is successfully updated	Student updated successfully	Pass
TC20	Send invalid student data directly through API	Backend rejects request	Invalid request rejected	Pass
TC21	Verify existing Delete functionality	Correct student is deleted	Delete worked correctly	Pass
TC22	Verify administrator authentication and authorization	Existing access control continues to work	Authentication/authorization worked correctly	Pass

3.3 Test Summary

Test Result	Count
Total Test Cases	22
Passed	22
Failed	0
Pass Rate	100%

Overall Test Result: Pass

4. Acceptance Criteria Verification

ID	Acceptance Criterion	Verification	Status
AC-001	Administrators cannot submit the form when required fields are empty.	Required field validation test	Met
AC-002	First names containing fewer than 3 characters are rejected.	First name validation test	Met
AC-003	Last names containing fewer than 3 characters are rejected.	Last name validation test	Met
AC-004	Names containing invalid numbers or inappropriate special characters are rejected.	Name character validation test	Met
AC-005	Invalid email addresses are rejected.	Email validation test	Met
AC-006	Phone numbers containing fewer than 10 digits are rejected.	Short phone number test	Met
AC-007	Phone numbers containing more than 10 digits are rejected.	Long phone number test	Met
AC-008	Phone numbers containing non-numeric characters are rejected.	Non-numeric phone test	Met
AC-009	A valid 10-digit phone number is accepted.	Valid phone test	Met
AC-010	Weak passwords are rejected when creating a new student.	Password validation test	Met
AC-011	A password containing at least 8 characters, uppercase, lowercase, number, and special character is accepted.	Strong password test	Met
AC-012	A valid student can be successfully created.	Student creation test	Met
AC-013	Invalid student information cannot be successfully created.	Invalid Add test	Met
AC-014	Invalid student information cannot be successfully updated.	Invalid Edit test	Met
AC-015	A valid student can be successfully updated.	Valid Edit test	Met
AC-016	Direct API requests containing invalid data are rejected by the backend.	Postman API validation test	Met
AC-017	Existing Edit and Delete functionality continues to affect the correct student.	CRUD regression test	Met
AC-018	Existing administrator authentication and authorization continue to work correctly.	Authentication and authorization regression test	Met

5. Evidence

A maximum of **two relevant screenshots** should be attached.

Evidence 01 – Student Form Validation

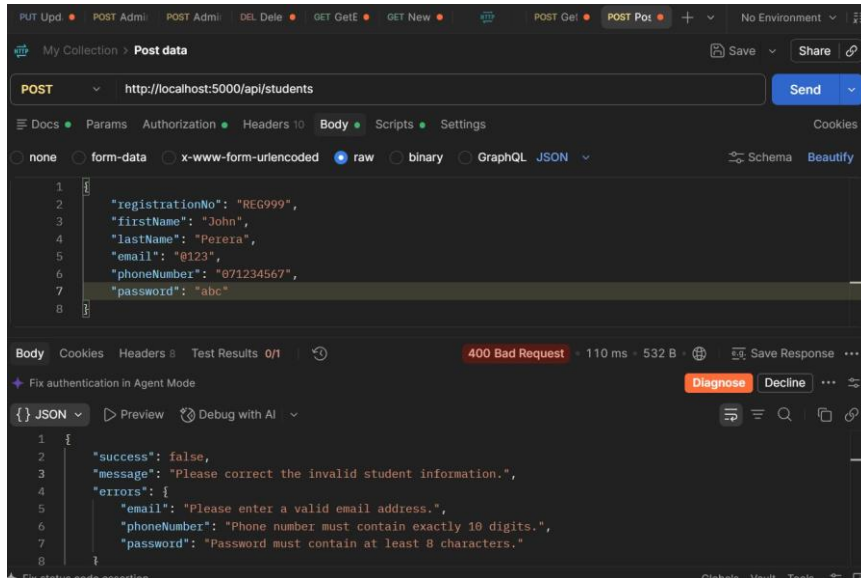
Related: AC-001, AC-002, AC-003, AC-004, AC-005, AC-006, AC-007, AC-008, AC-010, AC-011

The screenshot shows the 'Add New Student' modal form. A red error message at the top states: 'Please correct the highlighted fields before submitting.' The form fields are: Registration Number (REG006), First Name (Uga), Last Name (Yoga), Email (Uga123@gmail.com), and Phone Number (07512364). The Phone Number field has a red error message: 'Phone number must contain exactly 10 digits.' The Password field is empty and has a red error message: 'Password must contain at least one number, letter, and special character.'

The screenshot shows the 'Add New Student' modal form with the same data as the first screenshot. A red error message is displayed below the Email field: 'Please include an @ in the email address. Uga123 is missing an @.' The Email field contains 'Uga123'.

Evidence 02 – Backend Validation / Valid Student Operation

Related: AC-012, AC-013, AC-014, AC-015, AC-016, AC-017

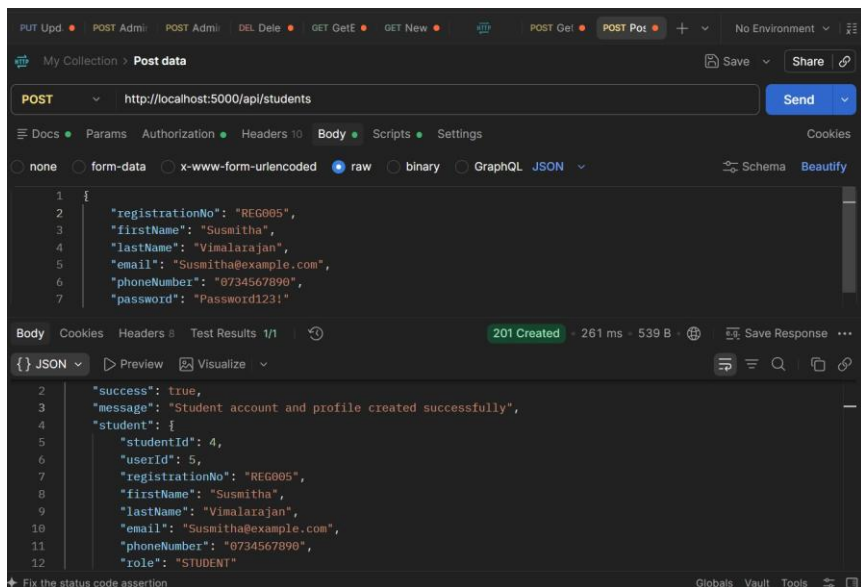


POST http://localhost:5000/api/students

```
1 {
2   "registrationNo": "REG999",
3   "firstName": "John",
4   "lastName": "Perera",
5   "email": "0123",
6   "phoneNumber": "071234567",
7   "password": "abc"
8 }
```

Body

```
1 {
2   "success": false,
3   "message": "Please correct the invalid student information.",
4   "errors": {
5     "email": "Please enter a valid email address.",
6     "phoneNumber": "Phone number must contain exactly 10 digits.",
7     "password": "Password must contain at least 8 characters."
8   }
9 }
```



POST http://localhost:5000/api/students

```
1 {
2   "registrationNo": "REG005",
3   "firstName": "Susmitha",
4   "lastName": "Vimalarajan",
5   "email": "Susmitha@example.com",
6   "phoneNumber": "0734567890",
7   "password": "Password123!"
8 }
```

Body

```
1 {
2   "success": true,
3   "message": "Student account and profile created successfully",
4   "student": {
5     "studentId": 4,
6     "userId": 5,
7     "registrationNo": "REG005",
8     "firstName": "Susmitha",
9     "lastName": "Vimalarajan",
10    "email": "Susmitha@example.com",
11    "phoneNumber": "0734567890",
12    "role": "STUDENT"
13  }
14 }
```

6. Final Result

Item	Result
Requirements	Met
Implementation	Completed
Testing	Passed
Acceptance Criteria	All Met
Existing CRUD	Verified
Authentication & Authorization	Verified
Password Security	Verified
Final Status	Accepted

6.1 Final Summary

CR-004 was implemented as a validation enhancement for the administrator student Add and Edit operations.

The implementation validates registration number, first name, last name, email address, phone number, and password according to the defined requirements. First and last names require a minimum of three characters and valid alphabetic characters. Email addresses must follow a valid email format, while phone numbers must contain exactly 10 numeric digits.

Strong password validation was implemented for new student accounts. Passwords must contain at least 8 characters, including uppercase and lowercase letters, a number, and a special character.

Both client-side and server-side validation were implemented. Client-side validation prevents invalid information from being submitted through the form, while server-side validation prevents validation bypass through direct API requests.

Existing Add, Edit, Delete, administrator authentication, authorization, database operations, API endpoints, and password hashing mechanisms were preserved.

The implemented functionality was tested using frontend validation, backend API testing, positive and negative test cases, and regression testing. All defined acceptance criteria were verified successfully.

6.2 Issues / Limitations

No significant issues identified.

The CR-004 requirements were implemented within the defined scope. No database schema redesign, new student management API endpoints, authentication changes, authorization changes, or unrelated module changes were required.

Student Name: Susmitha Vimalarajan

Student ID: JFICT24_30

Date: 08 October 2026

Signature: